```
ecommerce-app/
|
├── config/
|   ├── mongo.js
|   └── sql.js
|
├── controllers/
|   ├── productController.js
|   └── authController.js
|
├── models/
|   ├── product.mongo.js
|   └── product.sql.js
|
├── routes/
|   ├── productRoutes.js
|   └── authRoutes.js
|
├── services/
|   └── productService.js
|
├── middleware/
|   ├── authMiddleware.js
|   ├── authorize.js
|   ├── errorHandler.js
|   ├── logger.js
|   └── validateProduct.js
|
├── utils/
|   ├── apiResponse.js
|   └── AppError.js
|
├── .env
├── app.js
├── package.json
└── package-lock.json
```

App.js

```
const express = require("express");
require("dotenv").config();


// DB
const connectMongo = require("./config/mongo");
const { connectSQL } = require("./config/sql");
```

```
// Routes
const productRoutes = require("./routes/productRoutes");
const authRoutes = require("./routes/authRoutes");
```

```
// Middleware
const errorHandler = require("./middleware/authMiddleware");
const logger = require("./middleware/logger");
```

```
const app = express();
```

```
// Connect MongoDB
connectMongo();
```

```
// After Mongo connection
connectSQL();
```

```
// Middlewares
app.use(express.json());
app.use(logger);
```

```
// Routes
app.get("/", (req, res) => {
    res.send("API Running");
});
```

```
app.use("/api/products", productRoutes);
app.use("/api/auth", authRoutes);
```

```
// Error handler (must be last)
app.use(errorHandler);
```

```
// Start server
const PORT = process.env.PORT || 3000;
```

```
app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});
```

Config> mongo.js

```
const mongoose = require("mongoose");


const connectMongo = async () => {
    try {
        // Connect to MongoDB (no deprecated options)
        await mongoose.connect(process.env.MONGO_URI);

        console.log("MongoDB connected successfully");

        // Connection events
        mongoose.connection.on("connected", () => {
            console.log("Mongoose connected to DB");
        });

        mongoose.connection.on("disconnected", () => {
            console.log("Mongoose disconnected from DB");
        });

        mongoose.connection.on("error", (err) => {
            console.error("Mongoose connection error:", err.message);
        });

    } catch (error) {
        console.error("MongoDB Connection Failed:", error.message);
        process.exit(1);
    }
};

module.exports = connectMongo;
```

Config > sql.js

```
const { Sequelize } = require("sequelize");
require("dotenv").config();


// Create Sequelize instance
const sequelize = new Sequelize(
    process.env.DB_NAME,
    process.env.DB_USER,
```

```
    process.env.DB_PASSWORD,
    {
        host: process.env.DB_HOST,
        port: process.env.DB_PORT,
        dialect: "mysql",
        logging: false
    }
);
```

```
// Test connection
const connectSQL = async () => {
    try {
        await sequelize.authenticate();
        console.log("MySQL connected successfully");
    } catch (error) {
        console.error("MySQL connection failed:", error.message);
        process.exit(1);
    }
};
```

```
module.exports = { sequelize, connectSQL };
```

==========
=====

Routes > auth.js

```
// routes/authRoutes.js

const express = require("express");
const router = express.Router();
```

```
const { login } = require("../controllers/authController");
```

```
router.post("/login", login);
```

```
module.exports = router;
```

Routes > productRoutes.js

```
const express = require("express");
const router = express.Router();

const productController = require("../controllers/productController");
```

```
const validateProduct = require("../middleware/validateProduct");
const authMiddleware = require("../middleware/authMiddleware");
const authorize = require("../middleware/authorize");
```

```
// GET all products
router.get("/", productController.getAllProducts);
```

```
// GET product by ID
router.get("/:id", productController.getProductById);
```

```
// CREATE product
router.post(
    "/",
    validateProduct,
    authMiddleware,
    authorize("admin"),
    productController.createProduct
);
```

```
// UPDATE product
router.put(
    "/:id",
    validateProduct,
    authMiddleware,
    authorize("admin"),
    productController.updateProduct
);
```

```
// PATCH product
router.patch(
    "/:id",
    authMiddleware,
    authorize("admin"),
    productController.patchProduct
);
```

```
// DELETE product
router.delete(
    "/:id",
    authMiddleware,
    authorize("admin"),
    productController.deleteProduct
);
```

```
module.exports = router;
```

Middleware > authMiddleware.js

```
const jwt = require("jsonwebtoken");
const AppError = require("../utils/AppError");
```

```
const protect = (req, res, next) => {
    try {
        let token;

        // 1. Extract token from Authorization header
        if (
            req.headers.authorization &&
            req.headers.authorization.startsWith("Bearer ")
        ) {
            token = req.headers.authorization.split(" ")[1];
        }

        // 2. If no token
        if (!token) {
            return next(new AppError("Access denied. No token provided", 401));
        }

        // 3. Verify token
        const decoded = jwt.verify(
            token,
            process.env.JWT_SECRET || "mysecretkey"
        );

        // 4. Attach user info to request
        req.user = decoded;

        next();

    } catch (error) {

        // Handle JWT specific errors
        if (error.name === "JsonWebTokenError") {
            return next(new AppError("Invalid token", 401));
        }

        if (error.name === "TokenExpiredError") {
            return next(new AppError("Token expired", 401));
        }

        return next(new AppError("Authentication failed", 401));
    }
};

module.exports = protect;
```

Middleware > logger.js

```
const logger = (req, res, next) => {
```

```
    const method = req.method;
    const url = req.url;
    const time = new Date().toISOString();

    console.log(`${method} ${url} - ${time}`);

    next();
};

module.exports = logger;
```

Middleware > authorize.js

```
const AppError = require("../utils/AppError");


const authorize = (...allowedRoles) => {
    return (req, res, next) => {

        // 1. Ensure user exists (set by protect middleware)
        if (!req.user) {
            return next(new AppError("Unauthorized access", 401));
        }

        // 2. Check role
        if (!allowedRoles.includes(req.user.role)) {
            return next(new AppError("Forbidden: insufficient permissions", 403));
        }

        next();
    };
};

module.exports = authorize;
```

Middleware > validateProduct.js

```


const { errorResponse } = require("../utils/apiResponse");

const validateProduct = (req, res, next) => {
```

```
    const { name, price, category, stock } = req.body;
```

```
    // NAME
    if (typeof name !== "string" || name.trim() === "") {
        return errorResponse(res, "Product name must be a non-empty string", 400);
    }
```

```
    // PRICE
    if (typeof price !== "number" || isNaN(price) || price <= 0) {
        return errorResponse(res, "Price must be a number greater than 0", 400);
    }
```

```
    // CATEGORY
    if (typeof category !== "string" || category.trim() === "") {
        return errorResponse(res, "Category must be a non-empty string", 400);
    }
```

```
    // STOCK
    if (typeof stock !== "number" || isNaN(stock) || stock < 0) {
        return errorResponse(res, "Stock must be a non-negative number", 400);
    }
```

```
    next();
};
```

```
module.exports = validateProduct;
```

Models > product.mongo.js

```
const mongoose = require("mongoose");


const productSchema = new mongoose.Schema(
    {
        name: {
            type: String,
            required: true
        },
        price: {
            type: Number,
            required: true
        },
        category: {
            type: String
        },
        stock: {
            type: Number,
            default: 0
        }
    },
    {
```

```
        timestamps: true
    }
);
```

```
module.exports = mongoose.model("Product", productSchema);
```

Model > product.sql.js

```
const { DataTypes } = require("sequelize");
const { sequelize } = require("../config/sql");


const ProductSQL = sequelize.define(
    "Product",
    {
        id: {
            type: DataTypes.INTEGER,
            autoIncrement: true,
            primaryKey: true
        },
        name: {
            type: DataTypes.STRING,
            allowNull: false
        },
        price: {
            type: DataTypes.FLOAT,
            allowNull: false
        },
        category: {
            type: DataTypes.STRING
        },
        stock: {
            type: DataTypes.INTEGER,
            defaultValue: 0
        }
    },
    {
        tableName: "products",
        timestamps: true
    }
);
```

```
module.exports = ProductSQL;
```

Controllers > authController.js

```
// controllers/authController.js


const jwt = require("jsonwebtoken");
```

```
const { successResponse } = require("../utils/apiResponse");
const AppError = require("../utils/AppError");
```

```
// Mock users
const USERS = [
    {
        id: 1,
        username: "admin",
        password: "1234",
        role: "admin"
    },
    {
        id: 2,
        username: "user",
        password: "1234",
        role: "user"
    }
];
```

```
exports.login = (req, res, next) => {
    try {
        const { username, password } = req.body;
```

```
        // Validation
        if (!username || !password) {
            return next(new AppError("Username and password required", 400));
        }
```

```
        // Find user
        const user = USERS.find(u => u.username === username);
```

```
        if (!user || user.password !== password) {
            return next(new AppError("Invalid credentials", 401));
        }
```

```
        // Generate token
        const token = jwt.sign(
            {
                id: user.id,
                role: user.role
            },
            process.env.JWT_SECRET,
            { expiresIn: "1h" }
        );
```

```
        return successResponse(res, "Login successful", { token });
```

```
    } catch (error) {
        return next(new AppError("Login failed", 500));
    }
};
```

Controller > productController.js

```
const productService = require("../services/productService");
const { successResponse } = require("../utils/apiResponse");
const AppError = require("../utils/AppError");

// GET all products
exports.getAllProducts = async (req, res, next) => {
    try {
        const products = await productService.getAllProducts();
        return successResponse(res, "All products fetched successfully", products);
    } catch (error) {
        return next(new AppError(error.message || "Failed to fetch products", 500));
    }
};
```

```
// GET product by ID
exports.getProductById = async (req, res, next) => {
    try {
        const { id } = req.params;
```

```
        const product = await productService.getProductById(id);
```

```
        if (!product) {
            return next(new AppError("Product not found", 404));
        }
```

```
        return successResponse(res, "Product fetched successfully", product);
    } catch (error) {
        return next(new AppError("Invalid product ID", 400));
    }
};
```

```
// CREATE product
exports.createProduct = async (req, res, next) => {
    try {
        const product = await productService.createProduct(req.body);
        return successResponse(res, "Product created successfully", product, 201);
    } catch (error) {
        return next(new AppError(error.message || "Failed to create product", 500));
    }
};
```

```
// UPDATE product
exports.updateProduct = async (req, res, next) => {
    try {
        const { id } = req.params;
```

```
        const product = await productService.updateProduct(id, req.body);
```

```
        if (!product) {
```

```
            return next(new AppError("Product not found", 404));
        }

        return successResponse(res, "Product updated successfully", product);
    } catch (error) {
        return next(new AppError(error.message || "Failed to update product", 500));
    }
};

// PATCH product
exports.patchProduct = async (req, res, next) => {
    try {
        const { id } = req.params;

        const product = await productService.patchProduct(id, req.body);

        if (!product) {
            return next(new AppError("Product not found", 404));
        }

        return successResponse(res, "Product updated partially", product);
    } catch (error) {
        return next(new AppError(error.message || "Failed to patch product", 500));
    }
};

// DELETE product
exports.deleteProduct = async (req, res, next) => {
    try {
        const { id } = req.params;

        const product = await productService.deleteProduct(id);

        if (!product) {
            return next(new AppError("Product not found", 404));
        }

        return successResponse(res, "Product deleted successfully", product);
    } catch (error) {
        return next(new AppError(error.message || "Failed to delete product", 500));
    }
};
```

Services > productService.js

```
// const mongoose = require("mongoose");
// const productMongo = require("../models/product.mongo");
// const productSQL = require("../models/product.sql");
```

```
// // Enable both DBs
// const USE_MONGO = true;
// const USE_SQL = true;
```

```
// // GET all products
// const getAllProducts = async () => {
//     let mongoResult = null;
//     let sqlResult = null;
```

```
//     if (USE_MONGO) {
//         mongoResult = await productMongo.find();
//     }
```

```
//     if (USE_SQL) {
//         sqlResult = await productSQL.findAll();
//     }
```

```
//     if (USE_MONGO && USE_SQL) {
//         return {
//             mongo: mongoResult,
//             sql: sqlResult
//         };
//     }
```

```
//     if (USE_MONGO) return mongoResult;
//     if (USE_SQL) return sqlResult;
```

```
//     return [];
// };
```

```
// // GET product by ID
// const getProductById = async (id) => {
//     let mongoResult = null;
//     let sqlResult = null;
```

```
//     // Check if valid Mongo ObjectId
//     const isMongoId = mongoose.Types.ObjectId.isValid(id);
```

```
//     // Check if numeric (SQL ID)
//     const isSQLId = !isNaN(id);
```

```
//     // Mongo Query
//     if (USE_MONGO && isMongoId) {
//         mongoResult = await productMongo.findById(id);
//     }
```

```
//     // SQL Query
//     if (USE_SQL && isSQLId) {
//         sqlResult = await productSQL.findByPk(id);
//     }
```

```
//     // Return logic
//     if (USE_MONGO && USE_SQL) {
//         if (!mongoResult && !sqlResult) return null;
```

```
//         return {
//             mongo: mongoResult || null,
//             sql: sqlResult || null
//         };
//     }
```

```
//     if (USE_MONGO) return mongoResult;
//     if (USE_SQL) return sqlResult;
```

```
//     return null;
// };
```

```
// // CREATE product
// const createProduct = async (data) => {
//     let mongoResult = null;
//     let sqlResult = null;
```

```
//     if (USE_MONGO) {
//         mongoResult = await productMongo.create(data);
//     }
```

```
//     if (USE_SQL) {
//         sqlResult = await productSQL.create({
//             name: data.name,
//             price: data.price,
//             category: data.category,
//             stock: data.stock
//         });
//     }
```

```
//     if (USE_MONGO && USE_SQL) {
//         return {
//             mongo: mongoResult,
//             sql: sqlResult
//         };
//     }
```

```
//     if (USE_MONGO) return mongoResult;
//     if (USE_SQL) return sqlResult;
```

```
//     return null;
// };
```

```
// // UPDATE product (PUT)
// const updateProduct = async (id, data) => {
//     let mongoResult = null;
//     let sqlResult = null;
```

```
//     if (USE_MONGO) {
//         mongoResult = await productMongo.findByIdAndUpdate(
//             id,
//             data,
//             { new: true, runValidators: true }
//         );
//     }
```

```
//     if (USE_SQL) {
//         await productSQL.update(
//             {
//                 name: data.name,
//                 price: data.price,
//                 category: data.category,
//                 stock: data.stock
//             },
//             {
//                 where: { id: id }
//             }
//         );
```

```
//         sqlResult = await productSQL.findByPk(id);
//     }
```

```
//     if (USE_MONGO && USE_SQL) {
//         return {
//             mongo: mongoResult,
//             sql: sqlResult
//         };
//     }
```

```
//     if (USE_MONGO) return mongoResult;
//     if (USE_SQL) return sqlResult;
```

```
//     return null;
// };
```

```
// // PATCH product
// const patchProduct = async (id, data) => {
//     let mongoResult = null;
//     let sqlResult = null;
```

```
//     if (USE_MONGO) {
//         mongoResult = await productMongo.findByIdAndUpdate(
//             id,
//             data,
//             { new: true, runValidators: true }
//         );
//     }
```

```
//     if (USE_SQL) {
//         await productSQL.update(data, {
//             where: { id: id }
//         });
```

```
//         sqlResult = await productSQL.findByPk(id);
//     }
```

```
//     if (USE_MONGO && USE_SQL) {
//         return {
//             mongo: mongoResult,
//             sql: sqlResult
//         };
//     }
```

```
//     if (USE_MONGO) return mongoResult;
//     if (USE_SQL) return sqlResult;
```

```
//     return null;
// };
```

```
// // DELETE product
// const deleteProduct = async (id) => {
//     let mongoResult = null;
//     let sqlResult = null;
```

```
//     if (USE_MONGO) {
//         mongoResult = await productMongo.findByIdAndDelete(id);
//     }
```

```
//     if (USE_SQL) {
//         sqlResult = await productSQL.destroy({
//             where: { id: id }
//         });
//     }
```

```
//     if (USE_MONGO && USE_SQL) {
//         return {
//             mongo: mongoResult,
//             sql: sqlResult
//         };
//     }
```

```
//     if (USE_MONGO) return mongoResult;
//     if (USE_SQL) return sqlResult;
```

```
//     return null;
// };
```

```
// module.exports = {
//     getAllProducts,
//     getProductById,
//     createProduct,
//     updateProduct,
//     patchProduct,
//     deleteProduct
// };
```

```
const mongoose = require("mongoose");
const productMongo = require("../models/product.mongo");
const productSQL = require("../models/product.sql");
```

```
// Enable both DBs
const USE_MONGO = true;
const USE_SQL = true;
```

```
// Helpers
const isMongoId = (id) => mongoose.Types.ObjectId.isValid(id);
const isSQLId = (id) => !isNaN(id);
```

```
// -------------------- GET ALL --------------------
const getAllProducts = async () => {
    let mongoResult = null;
    let sqlResult = null;
```

```
    if (USE_MONGO) {
        mongoResult = await productMongo.find();
    }
```

```
    if (USE_SQL) {
        sqlResult = await productSQL.findAll();
    }
```

```
    if (USE_MONGO && USE_SQL) {
        return {
            mongo: mongoResult || [],
            sql: sqlResult || []
        };
    }
```

```
    if (USE_MONGO) return mongoResult;
    if (USE_SQL) return sqlResult;
```

```
    return [];
};
```

```
// -------------------- GET BY ID --------------------
const getProductById = async (id) => {
    let mongoResult = null;
    let sqlResult = null;
```

```
    if (USE_MONGO && isMongoId(id)) {
        mongoResult = await productMongo.findById(id);
    }
```

```
    if (USE_SQL && isSQLId(id)) {
        sqlResult = await productSQL.findByPk(id);
    }
```

```
    if (USE_MONGO && USE_SQL) {
        if (!mongoResult && !sqlResult) return null;
```

```
        return {
            mongo: mongoResult || null,
            sql: sqlResult || null
        };
    }
```

```
    if (USE_MONGO) return mongoResult;
    if (USE_SQL) return sqlResult;
```

```
    return null;
};
```

```
// -------------------- CREATE --------------------
const createProduct = async (data) => {
    let mongoResult = null;
    let sqlResult = null;
```

```
    if (USE_MONGO) {
        mongoResult = await productMongo.create(data);
    }
```

```
    if (USE_SQL) {
        sqlResult = await productSQL.create({
            name: data.name,
            price: data.price,
            category: data.category,
            stock: data.stock
        });
    }
```

```
    if (USE_MONGO && USE_SQL) {
        return {
            mongo: mongoResult,
            sql: sqlResult
        };
    }
```

```
    if (USE_MONGO) return mongoResult;
    if (USE_SQL) return sqlResult;
```

```
    return null;
};
```

```
// -------------------- UPDATE (PUT) --------------------
const updateProduct = async (id, data) => {
    let mongoResult = null;
    let sqlResult = null;
```

```
    if (USE_MONGO && isMongoId(id)) {
        mongoResult = await productMongo.findByIdAndUpdate(
            id,
            data,
            { new: true, runValidators: true }
        );
    }
```

```
    if (USE_SQL && isSQLId(id)) {
        const product = await productSQL.findByPk(id);
        if (product) {
            await product.update({
                name: data.name,
                price: data.price,
                category: data.category,
                stock: data.stock
            });
            sqlResult = product;
        }
    }
```

```
    if (USE_MONGO && USE_SQL) {
        if (!mongoResult && !sqlResult) return null;
```

```
        return {
            mongo: mongoResult || null,
            sql: sqlResult || null
        };
    }
```

```
    if (USE_MONGO) return mongoResult;
    if (USE_SQL) return sqlResult;
```

```
    return null;
};
```

```
// -------------------- PATCH --------------------
const patchProduct = async (id, data) => {
    let mongoResult = null;
    let sqlResult = null;
```

```
    if (USE_MONGO && isMongoId(id)) {
        mongoResult = await productMongo.findByIdAndUpdate(
            id,
            data,
            { new: true, runValidators: true }
        );
    }
```

```
    if (USE_SQL && isSQLId(id)) {
        const product = await productSQL.findByPk(id);
        if (product) {
            await product.update(data);
            sqlResult = product;
        }
```

```
    }

    if (USE_MONGO && USE_SQL) {
        if (!mongoResult && !sqlResult) return null;

        return {
            mongo: mongoResult || null,
            sql: sqlResult || null
        };
    }

    if (USE_MONGO) return mongoResult;
    if (USE_SQL) return sqlResult;

    return null;
};

// -------------------- DELETE --------------------
const deleteProduct = async (id) => {
    let mongoResult = null;
    let sqlResult = null;

    if (USE_MONGO && isMongoId(id)) {
        mongoResult = await productMongo.findByIdAndDelete(id);
    }

    if (USE_SQL && isSQLId(id)) {
        const product = await productSQL.findByPk(id);
        if (product) {
            await product.destroy();
            sqlResult = product;
        }
    }

    if (USE_MONGO && USE_SQL) {
        if (!mongoResult && !sqlResult) return null;

        return {
            mongo: mongoResult || null,
            sql: sqlResult || null
        };
    }

    if (USE_MONGO) return mongoResult;
    if (USE_SQL) return sqlResult;

    return null;
};

module.exports = {
    getAllProducts,
    getProductById,
```

```
    createProduct,
    updateProduct,
    patchProduct,
    deleteProduct
};
```

Utils > apiResponse.js

```
const successResponse = (res, message, data = null, statusCode = 200) => {
    return res.status(statusCode).json({
        success: true,
        message,
        data
    });
};

const errorResponse = (res, message = "Internal Server Error", statusCode = 500) => {
    return res.status(statusCode).json({
        success: false,
        message
    });
};
```

```
module.exports = {
    successResponse,
    errorResponse
};
```

Utils > AppError.js

```
class AppError extends Error {

    constructor(message, statusCode) {
        super(message);
```

```
        this.statusCode = statusCode;
```

```
        // Maintains proper stack trace (important for debugging)
        Error.captureStackTrace(this, this.constructor);
    }
}
```

```
module.exports = AppError;
```

.env

```
PORT=3000
JWT_SECRET=mysecretkey


MONGO_URI=mongodb://localhost:27017/ecommerce
SQL_DB=ecommerce
SQL_USER=root
SQL_PASSWORD=1234
SQL_HOST=localhost
```

```
DB_NAME=ecommerce
DB_USER=root
DB_PASSWORD=1234
DB_HOST=localhost
DB_PORT=3306
```